TinyML Studio

User Manual

Version 0.0.1

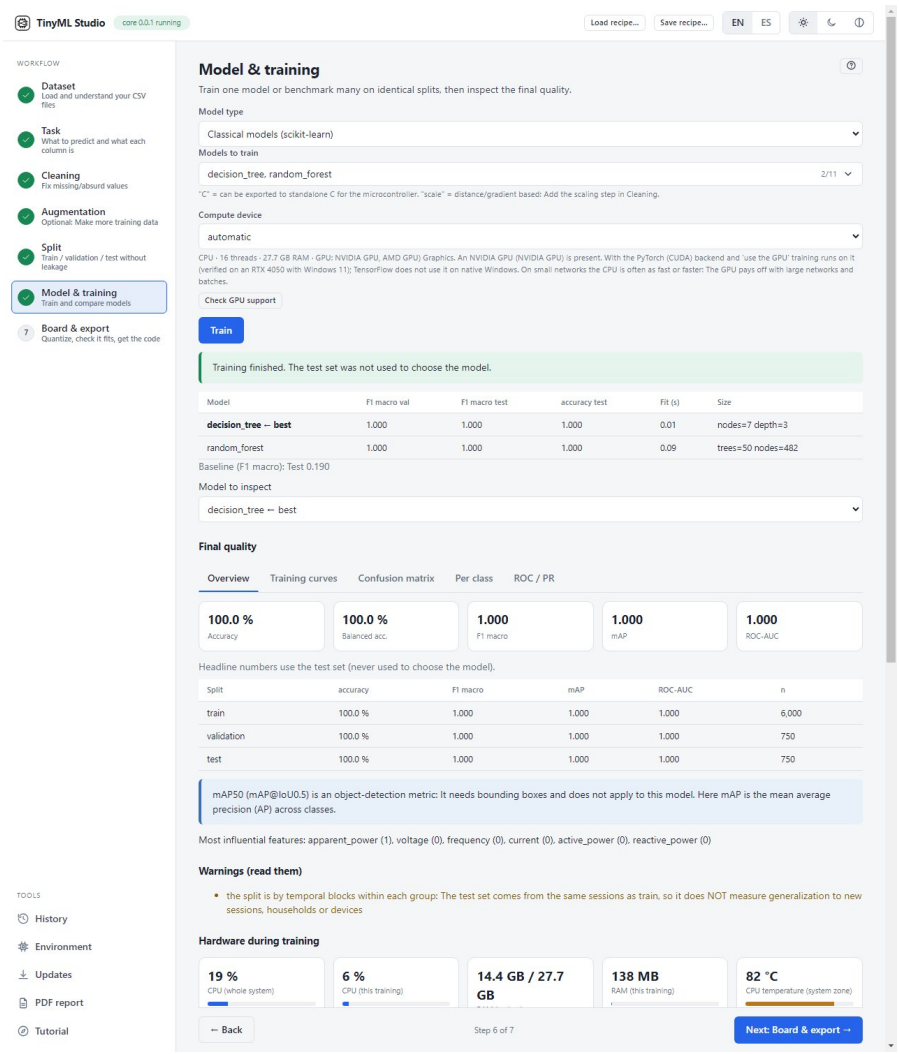


Figure 1. The model and training step, where most of the work happens.

Turn sensor data into a model that fits on your microcontroller.

Free software under the GNU General Public License v3.0.
Copyright (C) 2026 BrianGTRxx. Developed with the assistance of Claude (Anthropic).

Contents

1. What TinyML Studio is 3
2. Installing and first run 4
3. The main window 6
4. Step 1: Dataset 7
5. Step 2: Task 9
6. Step 3: Cleaning 13
7. Step 4: Augmentation 14
8. Step 5: Split 16
9. Step 6: Model and training 18
10. Step 7: Board and export 22
11. Using the model on an ESP32-S3 25
12. Tools: history, reports and the terminal 26
13. Troubleshooting 28
14. What is verified and what is not 28
15. Glossary 29
16. License and credits 30

1. What TinyML Studio is

TinyML Studio is a desktop application that guides you from a folder of sensor recordings to a small machine learning model that runs on a microcontroller such as the ESP32-S3. It takes care of the parts that are easy to get wrong: cleaning the data, splitting it without leaking information, training several models, shrinking the result, and checking that it fits in the memory of your board before you flash anything.

1.1 Concepts you need first

Machine learning model

A mathematical function that was fitted to examples. You show it many rows of inputs together with the correct answer, and it learns to produce the answer for inputs it has never seen.

TinyML

Machine learning on devices with very little memory and no operating system, such as a microcontroller. The model must be small (kilobytes), fast, and must not need a network connection.

Feature

One input column the model reads, for example the current, the active power or the power factor.

Target

The thing you want to predict. If it is a category (the name of an appliance) the problem is classification. If it is a number (an exam score) the problem is regression.

Flash and RAM

A microcontroller keeps the program and the model constants in flash (permanent, large) and the working values in RAM (temporary, small). A model that does not fit in either cannot run on the board.

Training, validation and test sets

The data is divided into three parts. The model learns from the training part, you compare models and tune settings on the validation part, and the test part is looked at once, at the end, to report an honest result. Looking at the test part to make decisions makes the result look better than it is.

Data leakage

Any way in which information from the validation or test data reaches the training process. Leakage produces beautiful numbers that disappear in the real world. TinyML Studio is built to avoid the common forms of it, and warns you about the ones it cannot prevent.

1.2 The seven steps

The left sidebar shows the workflow. A step is unlocked when the one before it is complete.

Step	Name	What you do there
1	Dataset	Open your CSV files and let the app understand them.
2	Task	Say what to predict and what each column is.

3	Cleaning	Fix missing and absurd values.
4	Augmentation	Optional: create new training rows from your real rows.
5	Split	Divide the data into training, validation and test.
6	Model and training	Train one model or compare many, and check the quality.
7	Board and export	Pick your board, check that it fits and take the code.

1.3 What it produces

At the end you obtain C code you can compile into your firmware, optionally a TensorFlow Lite file, an ONNX file, and a ready to open Arduino sketch. Every export is compared with the original model on your test data, so you know the version on the board gives the same answers as the one you trained.

> Note. Everything runs on your computer. Your data is never uploaded anywhere. The only network use is downloading Python packages when you ask the app to install them.

2. Installing and first run

2.1 Installing

Download the installer from the Releases page of the project. On Windows run the Setup program, or unzip the zip version and start TinyML Studio.exe. On Linux make the AppImage executable and run it, or install the deb package. Python and Node.js are not required to use the installed application.

> Warning. The Windows installer is not code signed. Windows SmartScreen may show a warning the first time. Choose More info, then Run anyway, if you trust the download.

2.2 New versions

The first time you start the application it asks once whether you want to be told when a new version is available. If you say yes, it asks GitHub, about once a day, whether a newer release exists. Nothing about you or your files is sent. If a newer version exists, a notice appears at the top with three choices: See what is new and download, which opens the download page in your browser, Not now, and Skip this version. The application never downloads or installs anything by itself, so you always decide.

The page Updates, in the sidebar, shows your version and a button Check for updates that you can press at any time, with or without the automatic notice. To update, run the new installer over the old one. It replaces the program and keeps your settings. There is no need to uninstall first.

2.3 The Python environment

The application is small because it does not include the scientific Python packages. The first time you start it, it needs a Python environment. There are two ways to get one, and the Environment page (in the sidebar, under Tools) explains both.

Option 1, install for me (recommended)

The app downloads its own private copy of Python and only the components you tick: the core (always), the terminal interface, Keras with TensorFlow, PyTorch, and TFLite Micro. Nothing else on your computer is touched. Keras and PyTorch with GPU support are large, about two gigabytes each, so tick them only if you need them.

Option 2, use my own environment

You point the app to a Python you already have (a virtual environment, conda or miniconda). The app checks what is missing and, after asking you, installs only the missing pieces. Packages that already satisfy the required range are not touched.

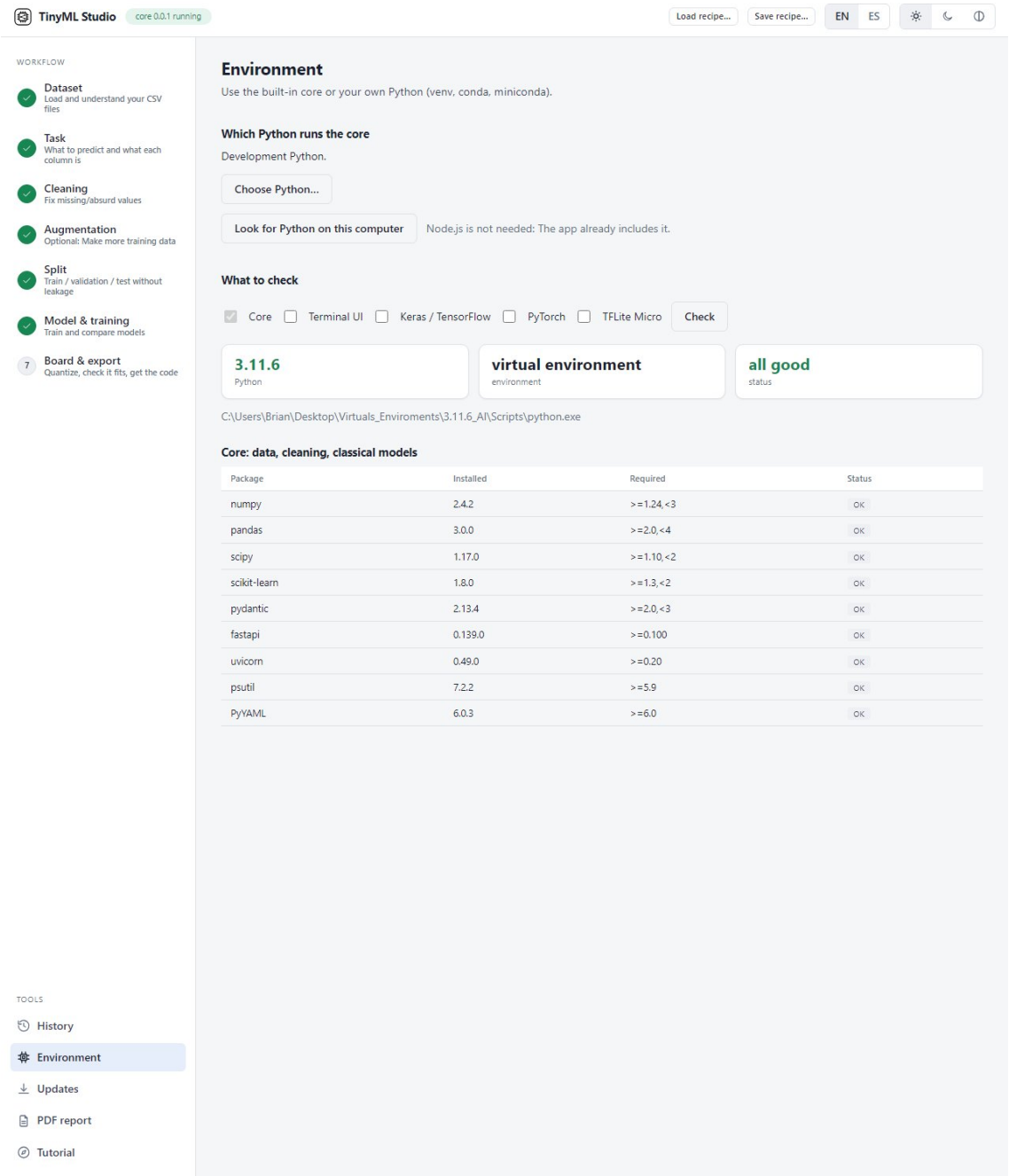


Figure 1. The Environment page. Look for Python on this computer lists the interpreters already installed.

The button Look for Python on this computer scans your PATH and the usual install locations, and shows each Python with a verdict. Versions 3.11 and 3.12 are recommended, 3.10 works, anything older than 3.10 cannot be used, and 3.13 or newer is marked as newer than tested

because some libraries, TensorFlow in particular, may not support it yet.

> Important. The app never upgrades or changes your Python. Adding packages to an environment you already use can change library versions that your other projects depend on. A clean virtual environment, or the private Python that the app installs, avoids that risk.

What if you choose both? Only one environment runs the core at a time. Choosing your own environment replaces the private one until you press Stop using my environment.

3. The main window

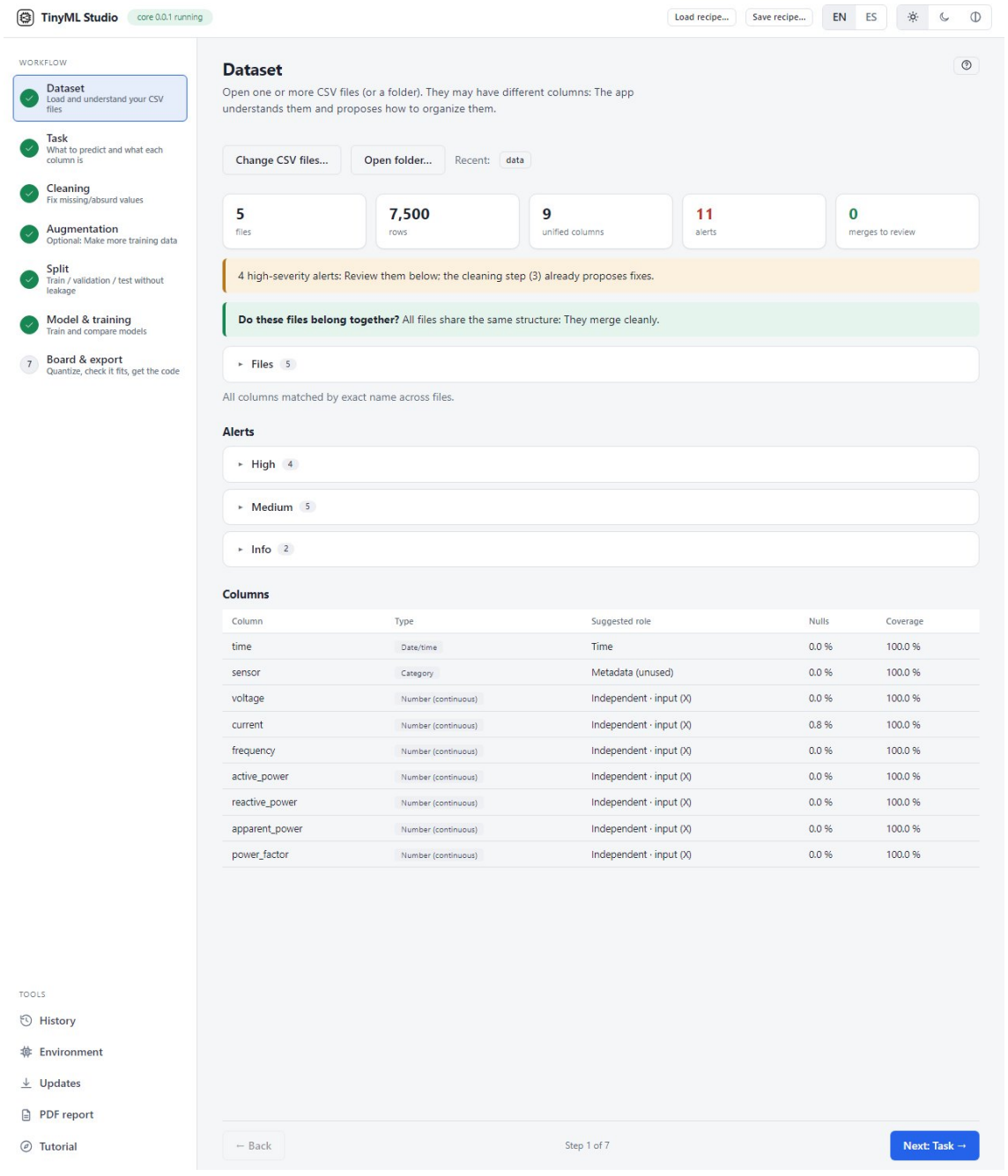


Figure 2. The main window: workflow on the left, the current step in the center, navigation at the bottom.

The sidebar lists the seven steps. Below it are the tools: History (past runs), Environment, PDF

report and Tutorial (the guided tour). The top bar has Load recipe and Save recipe, the language switch (English or Spanish), and the theme switch (light, dark or follow the system). The bar at the bottom moves between steps.

3.1 Recipes

Everything you configure is stored in a recipe, a small JSON file. Save recipe writes it, and Load recipe restores it, so you or another person can repeat exactly the same run on the same or another dataset. Recipes contain settings and file paths, not your data.

3.2 Language and theme

You can switch language at any time. Messages produced by the core, such as warnings and reports, are translated together with the interface, and results that are already on screen are translated without running them again. Names of columns and your own data are never translated.

4. Step 1: Dataset

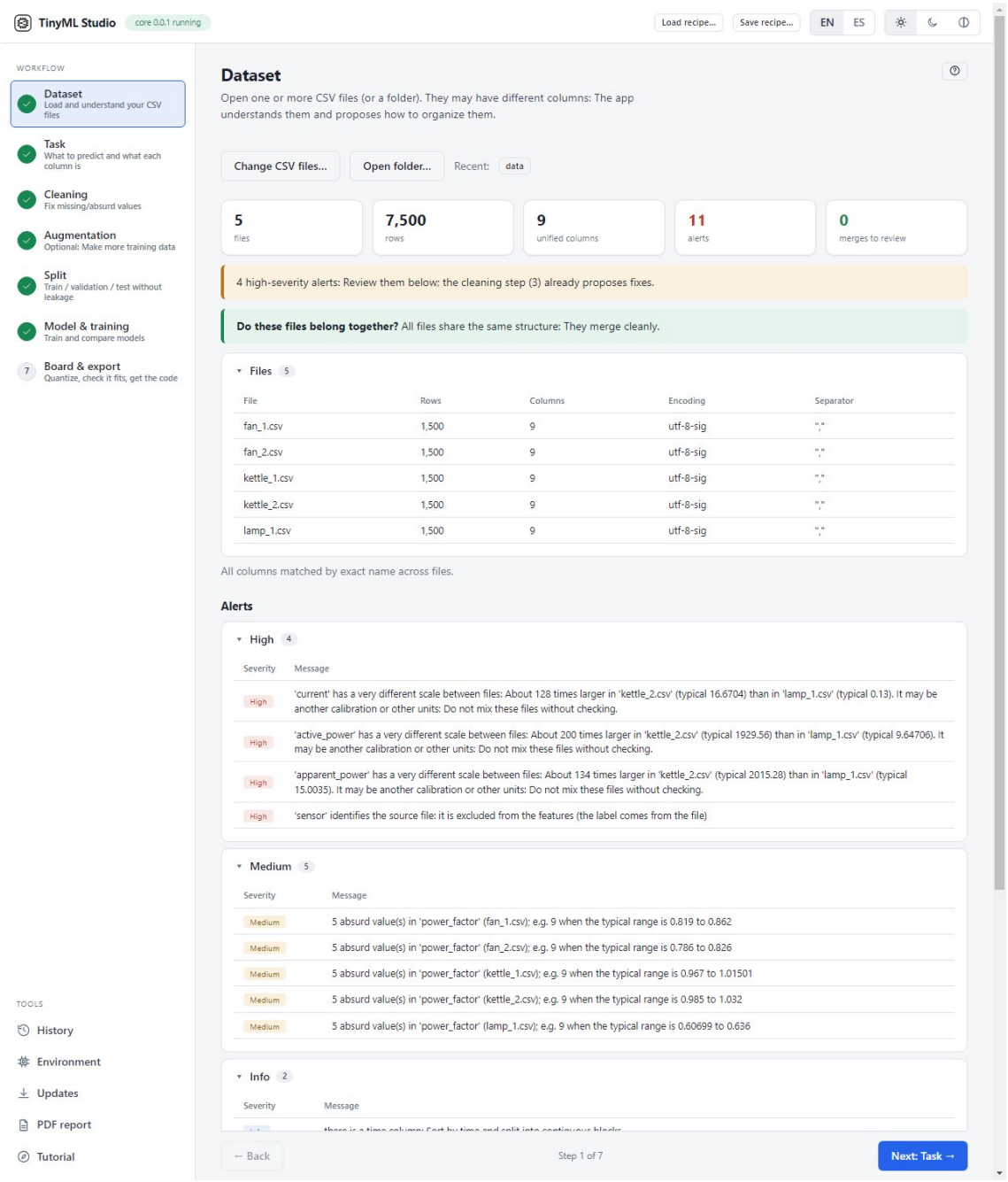


Figure 3. Step 1 after opening a folder of recordings, with the file table and quality alerts.

This step loads your CSV files. The application reads files exported from InfluxDB, files that use a comma as the decimal mark, and files in the latin-1 encoding, and it detects the separator and the encoding by itself.

4.1 Opening data

Open CSV

Opens the system file dialog. Select one file, or several with Ctrl or Shift. Use this when you want exactly some files out of a folder. Once data is loaded the same button is called Change CSV files.

Open folder

Takes every CSV in the folder. Use this when all the files belong to the project.

Recent

Under the buttons, the last data sets you opened appear as small buttons. One click opens the same files again, and the app checks that they still exist.

If you pick files that have different columns, the app merges columns with the same or similar names, and shows the groups it merged so you can check them. A column that exists in only some of the files is marked as missing for the others.

4.2 What you see

1. The number of files, rows and unified columns.
2. A card that tells you whether the files belong together. Same structure means they merge cleanly. Similar means some columns exist only in some files. Different kinds of data (for example electrical measurements and soil quality) are reported as groups, and you can continue with one group only.
3. The file table with rows, columns, encoding and separator of each file.
4. Quality alerts sorted by severity: columns with many missing values, absurd values, sentinel values, duplicated rows, and columns that identify the file of origin. An alert also appears when two files seem measured differently: a column with a very different scale (another calibration or other units) or with the opposite sign (another sign convention).

> If you ignore the compatibility card. Nothing is blocked. But a model trained on a mix of unrelated measurements usually learns the difference between the files and not what you care about. Mix files only when the study needs it.

> Tip. A column that identifies the recording, such as a sensor name that is different in every file, is flagged. If it stays as an input, the model can recognise the file instead of the appliance, and the results look better than they are.

5. Step 2: Task

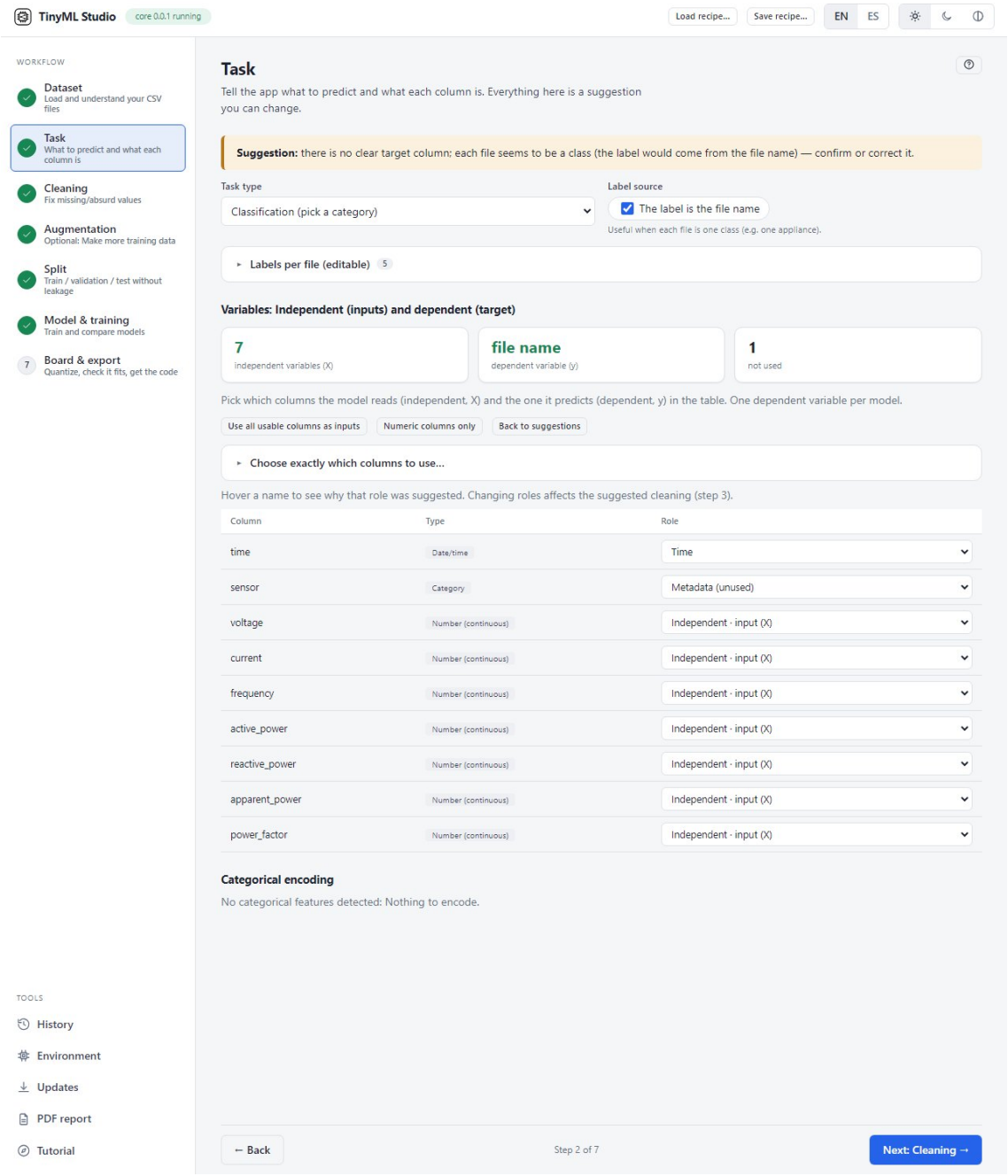


Figure 4. Step 2: the task type, the label source and the table of columns with their roles.

Here you tell the application what to predict and what each column means. Everything shown is a suggestion that you can change.

5.1 Task type

Classification

The answer is a category, such as kettle, fan or lamp. The model outputs the category. Choose this to recognise appliances.

Regression

The answer is a number, such as an exam score or a temperature. Choose this to estimate a value.

No label

There is nothing to learn from, so supervised models cannot be trained. Use the anomaly detector in step 6, or choose a target.

If you choose classification but the target column holds a continuous number, the classes would be thousands of different values and the result would be meaningless. If you choose regression for a category stored as text, the app cannot use it as a number. When in doubt, follow the suggestion at the top of the page.

5.2 Label source

Turn on The label is the file name when each file contains one class, for example one file per appliance. Numbers and suffixes in the file name are removed, so kettle_1.csv and kettle_2.csv both become the class kettle. Leave it off when a column of the table holds the class, and pick that column as the dependent variable.

5.3 Roles of the columns

Role	Meaning	Effect
Independent, input (X)	A column the model reads.	Becomes an input of the model and of the firmware.
Dependent, target (y)	The column to predict.	Only one per model.
Group	Identifies a recording or session.	Used to split by group and to validate on unseen sessions.
Time	A timestamp.	Used for temporal splits. Never an input.
Metadata (unused)	Information you keep but do not use.	Ignored by the model.
Discard	Remove the column.	Ignored and removed from the data.

The quick buttons help with many columns. Use all usable columns as inputs makes every numeric and categorical column an input. Numeric columns only keeps numbers. Back to suggestions restores the proposal.

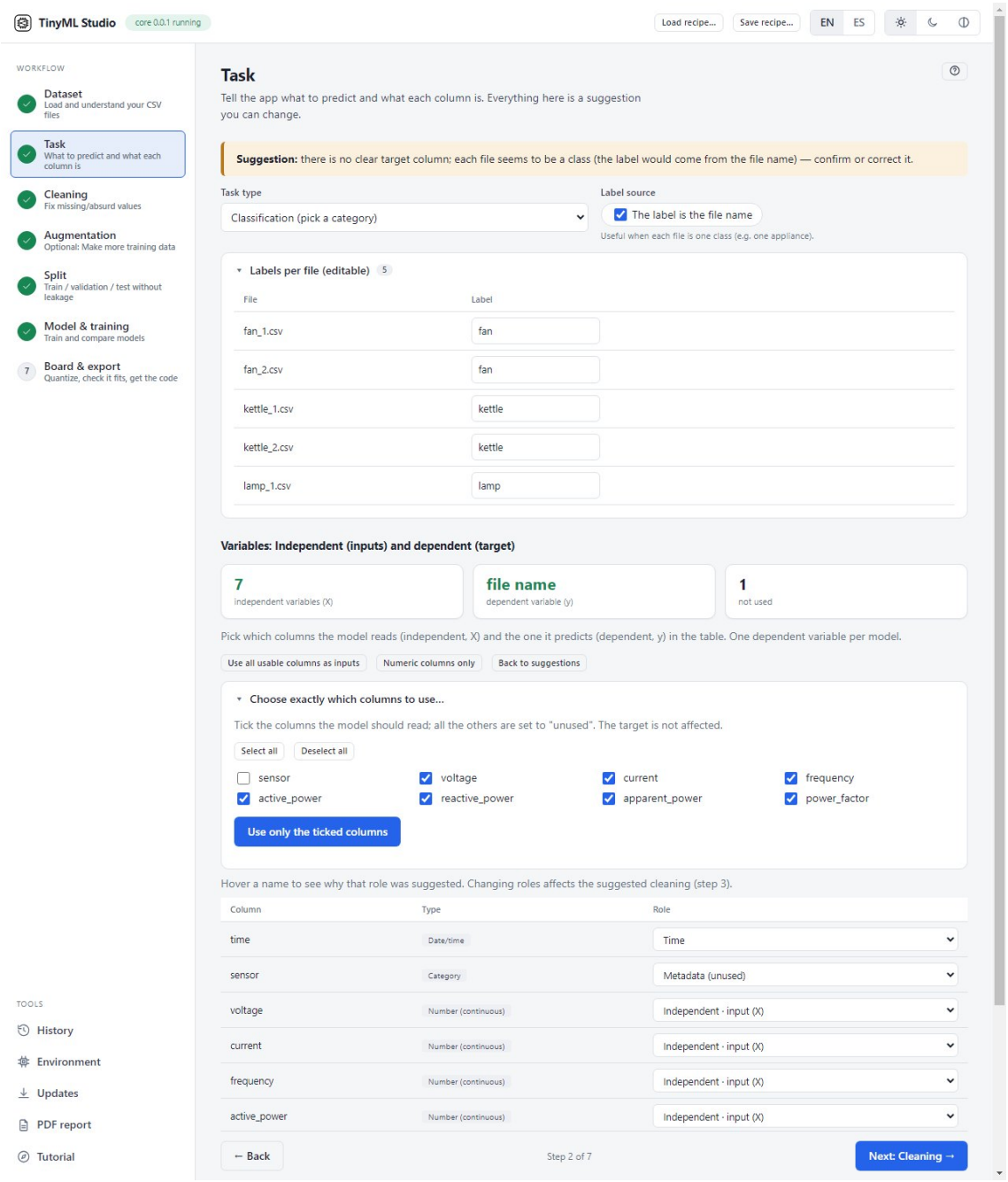


Figure 5. Choose exactly which columns to use: tick only the inputs you want.

Choose exactly which columns to use opens a list of checkboxes. Tick the columns the model must read and press Use only the ticked columns. Every other usable column becomes unused, and the target is not touched.

> If you select a column but not its relatives. Every input you keep must also be available on the board. If an input is computed by your firmware from other readings, make sure you can compute the same value there. If you remove an input, the exported code no longer asks for it.

5.4 Encoding of categories

A model reads numbers. A column of categories (for example Low, Medium, High) is converted. One-hot creates one yes or no column per category. Dummy is the same without one redundant column. Ordinal uses one number per category and suits categories that have a natural order. The limit of categories per column protects you from turning an identifier into

hundreds of columns.

6. Step 3: Cleaning

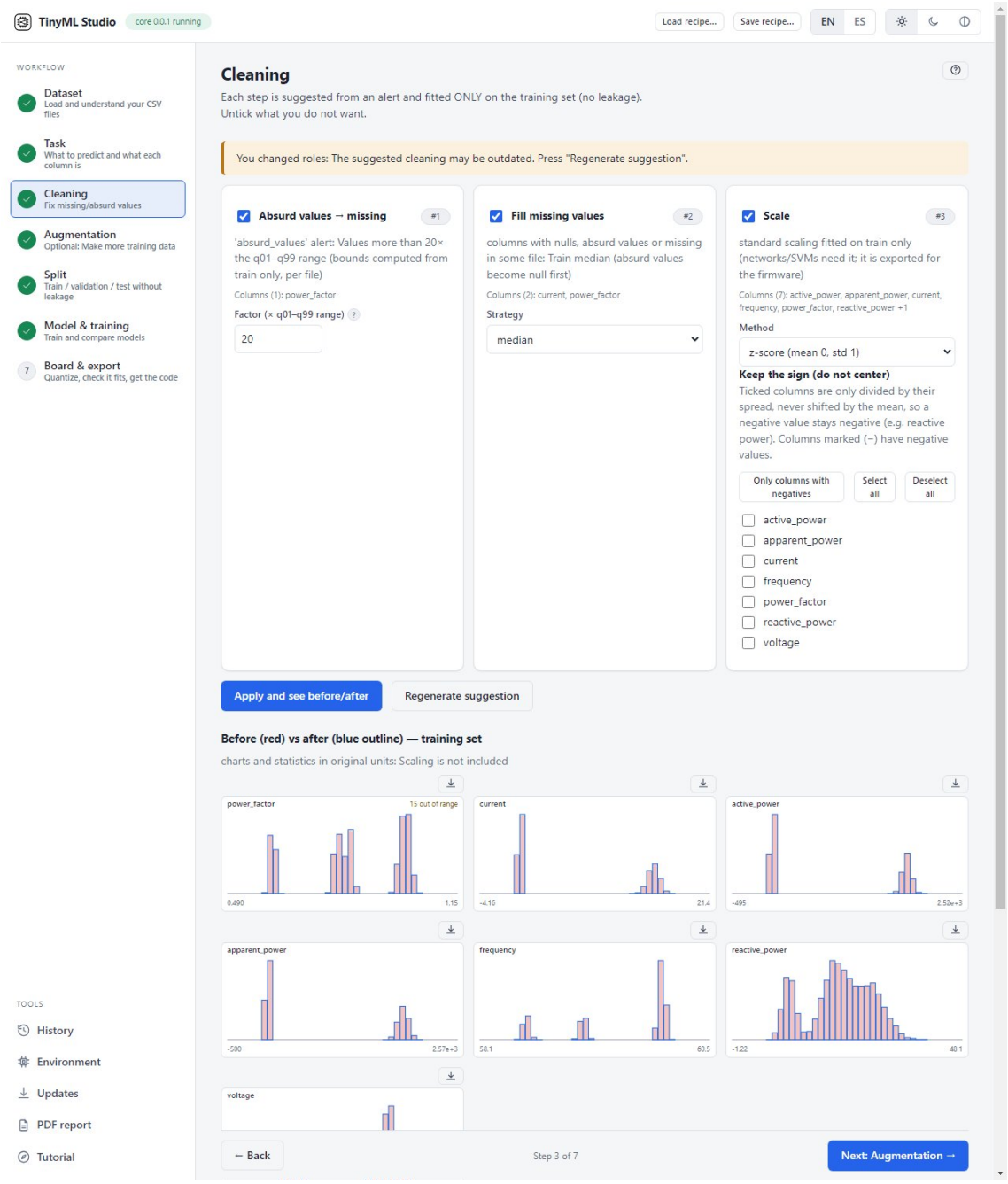


Figure 6. Step 3: each cleaning step comes from an alert and can be unticked or adjusted.

Real recordings contain gaps and impossible values. This step proposes a list of cleaning operations, each one created from a quality alert, and lets you change them.

Operation	What it does	When to keep it
Absurd values to missing	Marks values far outside the usual range as missing.	Almost always. A power factor of 9 is a reading error.

Fill missing values	Replaces gaps by the median, mean, mode or the previous value.	When a few values are missing. Median is robust.
Scale	Rescales columns (z-score, min-max or robust).	For neural networks and support vector machines. Trees do not need it.
Clip outliers	Limits values to a range.	When rare extreme values distort the model.
Replace sentinel values	Turns codes such as -999 into missing.	When your instrument writes such codes.
Remove duplicates	Deletes repeated rows, in training only.	When the logger repeats rows.

6.1 No leakage

Every number a step needs (the median, the mean, the limits) is computed from the training set only, and then applied unchanged to validation and test. This is why you can trust the validation numbers. The same numbers are exported to the firmware, so the board cleans its inputs exactly as the training did.

> Important. The column you want to predict is never scaled. In a regression, the model and the firmware answer in real units (points, degrees), not in standardized values.

6.2 If you change roles after the suggestion

When you change a role in step 2, a warning says the suggestion may be outdated. Press Regenerate suggestion to recompute it. If you do not, steps may still refer to columns you no longer use.

Apply and see before and after runs the cleaning and shows charts for each column. Every chart can be downloaded as PNG, JPG or SVG with the button above it. Statistics are shown in original units.

7. Step 4: Augmentation

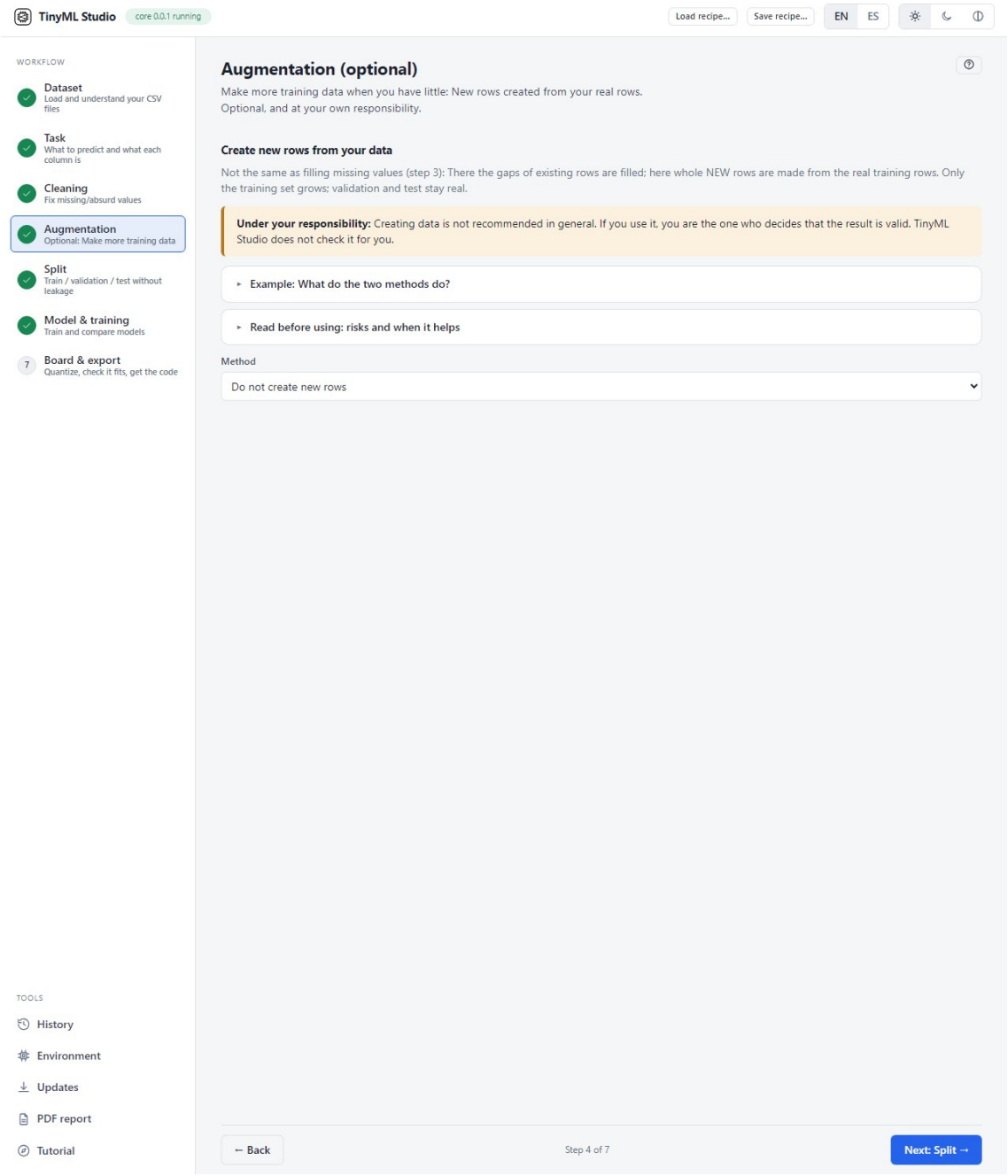


Figure 7. Step 4 is optional: it creates new training rows from your real rows, at your own responsibility.

Augmentation means making new rows from the rows you already have, when you have few. It is not the same as filling missing values in step 3: there the gaps of existing rows are filled, here whole new rows are made.

> Under your responsibility. Creating data is not recommended in general. If you use it, you are the one who decides that the result is valid. TinyML Studio does not check it for you.

7.1 The two methods

Copies with a little noise

Each new row is a real row plus a small random noise in every numeric column. The noise is a fraction of the deviation of that column. The new rows stay very close to real ones.

Between neighbouring rows

Each new row lies between a real row and one of its closest neighbours of the same class. For a regression, the target is interpolated too. The idea is the same as SMOTE.

You can ask for a number of new rows per real row, or, for classification, only as many as are needed to equalize the classes. Only the training set grows. Validation and test always keep the real rows.

7.2 An example

You have 3 real students (study hours, score): A (10, 60), B (12, 66) and C (30, 90). With noise, A can become (10.3, 59.4) and then (9.8, 60.5). Between neighbours, A and B are close, so a new student (11, 63) appears between them, while C is far from both and is not mixed with them.

7.3 The risks

1. New rows add no information that is not already in your data.
2. They can make a model look better than it is. Trust only validation and test, which stay real. Cross-validation inside training becomes optimistic.
3. Noise or interpolation can create rows that could never happen in real life.
4. Outliers and mistakes in your data are copied too. Clean first, in step 3.
5. Interpolating between classes that overlap blurs the border between them.
6. When you can, measuring more real data is always better than creating new rows.
7. It helps most with very few rows, strongly unbalanced classes, or a model that memorizes the training set.

What if you skip it? Nothing is lost. The rest of the workflow is the same, with real rows only.

8. Step 5: Split

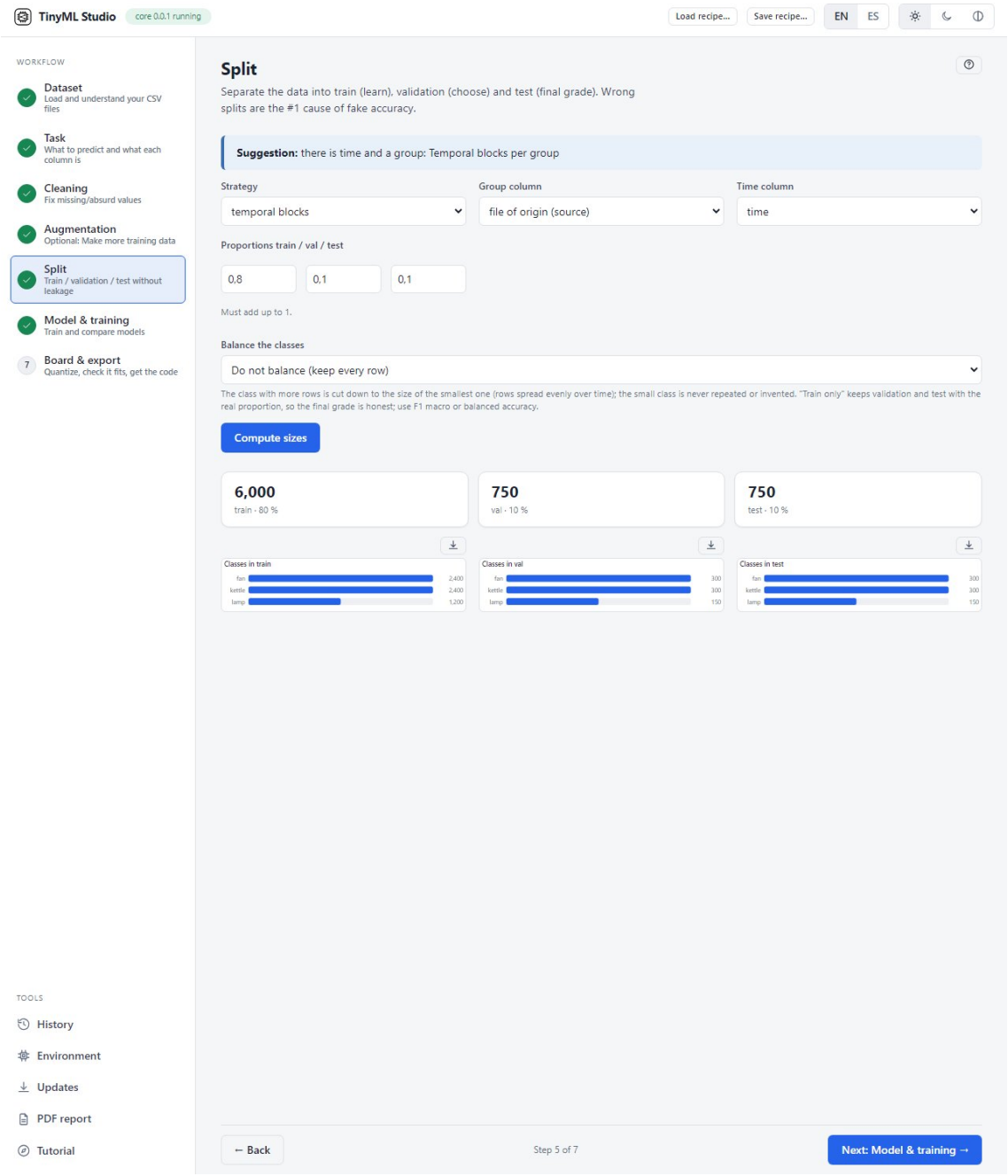


Figure 8. Step 5: the split strategy and the size of each part.

The data is divided into training (80 percent by default), validation (10 percent) and test (10 percent). How you divide matters more than most settings.

Strategy	How it divides	Use it when
Temporal blocks	Keeps time order: earlier rows train, later rows validate and test, inside each group.	Data recorded in sequence. Consecutive rows are almost identical, so a random split would leak.
By group	Whole recordings go to one part.	You have several recordings and want to test on unseen ones.
Stratified	Random, but every class keeps its proportion.	Rows are independent and classes are unbalanced.

Random	Pure random rows.	Rows are independent and the classes are balanced.

> If you pick random for time series. Neighbouring rows are nearly copies of each other. A random split puts some of them in training and their twins in test, and the accuracy looks excellent for the wrong reason. Follow the suggestion unless you know why not.

Check the sizes and the class distribution of every part. A class with fewer than twenty rows in the test part gives unreliable metrics, and the app warns you.

9. Step 6: Model and training

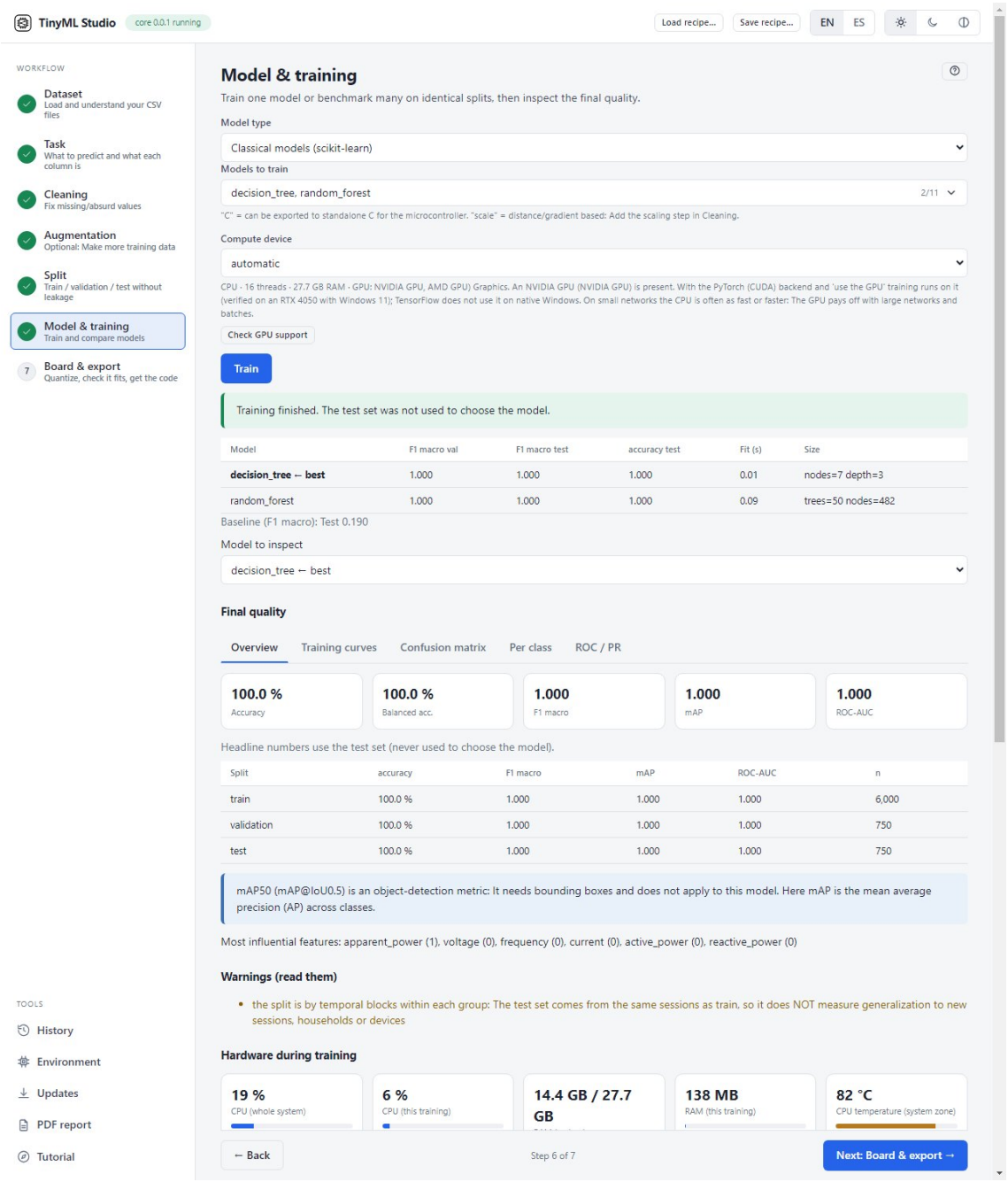


Figure 9. Step 6: choose the model family, the models, the compute device and press Train.

9.1 Choosing models

Classical models (scikit-learn)

Decision tree, random forest, extra trees, gradient boosting, logistic regression, linear models, support vector machines, k nearest neighbours, naive Bayes and a small neural network. They train in seconds, are easy to export to C, and are often enough for sensor data.

Keras MLP and 1D CNN

Neural networks. They need TensorFlow or PyTorch (installed from the Environment page) and export to TensorFlow Lite. A 1D CNN reads windows of consecutive samples.

Benchmark

Trains many models on identical splits and ranks them without looking at the test set.

Each model in the list shows badges. C means it can be exported to plain C. Scale means it works better after scaling. If you pick a model that cannot be exported to C, you can still train and compare it, but you cannot flash it as C code.

9.2 Compute device

Automatic uses a graphics card when the Python environment can see one (NVIDIA with CUDA, or AMD where supported) and the processor otherwise. CPU forces the processor. Classical scikit-learn models always use the CPU. Small networks are often as fast on the CPU as on a GPU. The page Support for GPU on this computer tells you what was found and how to enable it.

9.3 Reading the results

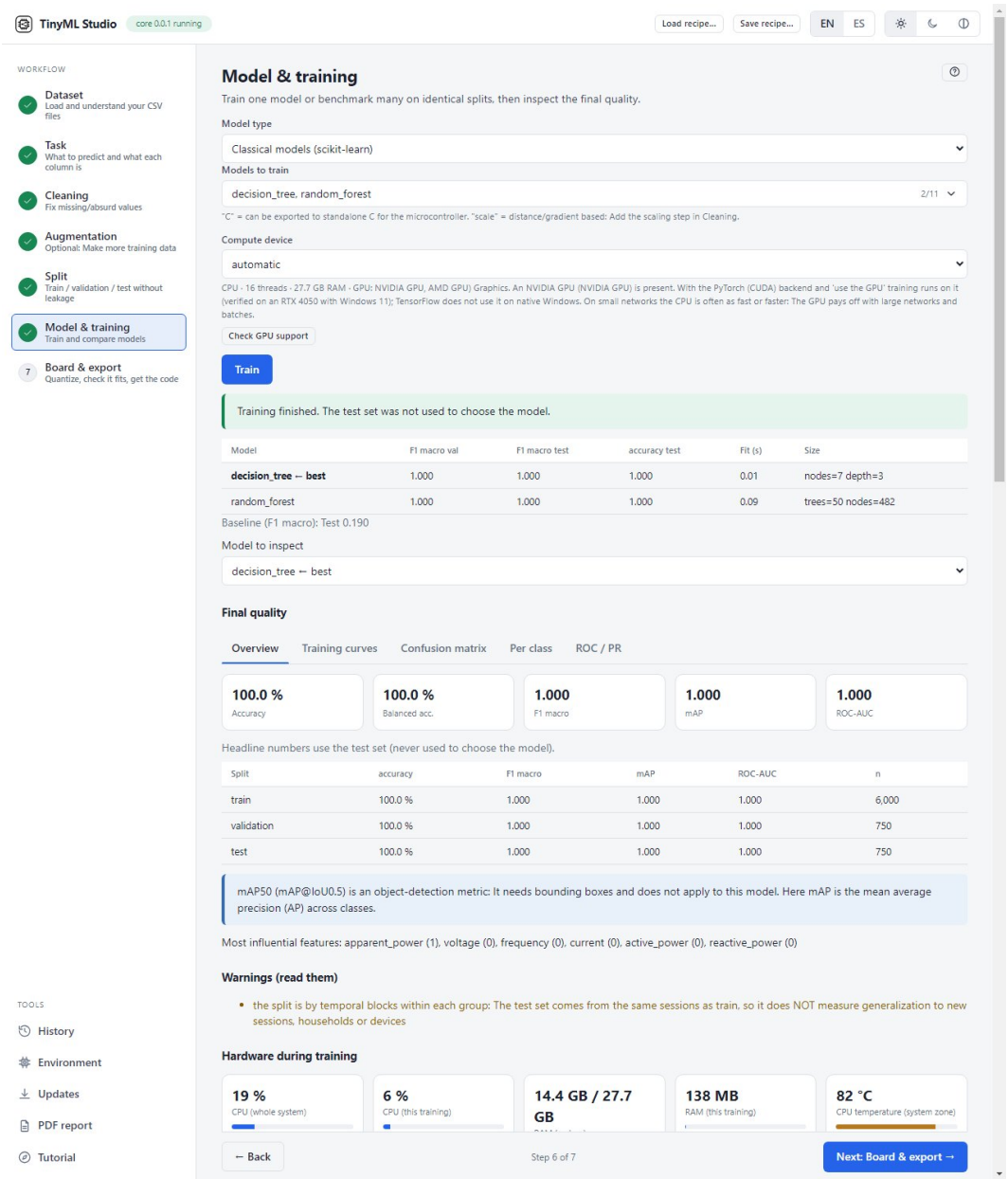


Figure 10. Step 6 with the extra cards expanded: unseen sessions, settings that fit the board, anomalies.

F1 macro

The average over all classes of a score that balances precision and recall. It treats every

class equally, so a rare class counts as much as a common one. It is the main number for classification.

Accuracy
The fraction of correct answers. It can mislead when one class is much larger than the others.

Confusion matrix
A table of what the model answered against what was true. The diagonal is correct.

R2 and MAE (regression)
R2 is how much of the variation the model explains, 1 is perfect. MAE is the average error in the units of the target.

Baseline
The score of always answering the most common class. A model must beat it clearly.

The tabs of the quality panel are Overview, Training curves, Confusion matrix, Per class and ROC / PR. A model that scores far higher in training than in validation is overfitting, which means it memorised.

9.4 Does it work on a new session?

The normal test comes from the same recordings as the training, so it measures how the model behaves later in the same capture. This card leaves one whole recording out, trains on the others, and measures on the recording the model never saw. A class can only be measured when it exists in at least two recordings.

If the card reports that your data cannot tell you how the model behaves on a new session, it is true: with one recording per class, a high score cannot separate the appliance from the conditions of that recording. The button Mark session columns as metadata and repeat removes columns that identify the recording, and shows the result before and after.

9.5 Settings that fit my board

This card tries many settings of one model, measures how much flash and RAM each would use on the board with the same calculation as the export, discards those that do not fit, and keeps the most accurate that does. The choice uses validation only, and the test set is measured once for the winner. Use these settings when training passes them to the next training.

9.6 Detect anomalies

The anomaly detector learns only what is normal and raises an alarm for anything that does not look like it. Mark the classes that are normal. The other classes are used only to measure how many it catches, never to train or to set the threshold. If you mark none, every row counts as normal and only false alarms are measured. A fault the detector has never seen may go unnoticed.

9.7 The hardware panel

While training, the panel shows processor and memory load, GPU load, GPU memory and temperatures, a chart of the last seconds, and the estimated time left. Sensors your system does not expose are shown as not available, with the reason, instead of invented values.

10. Step 7: Board and export

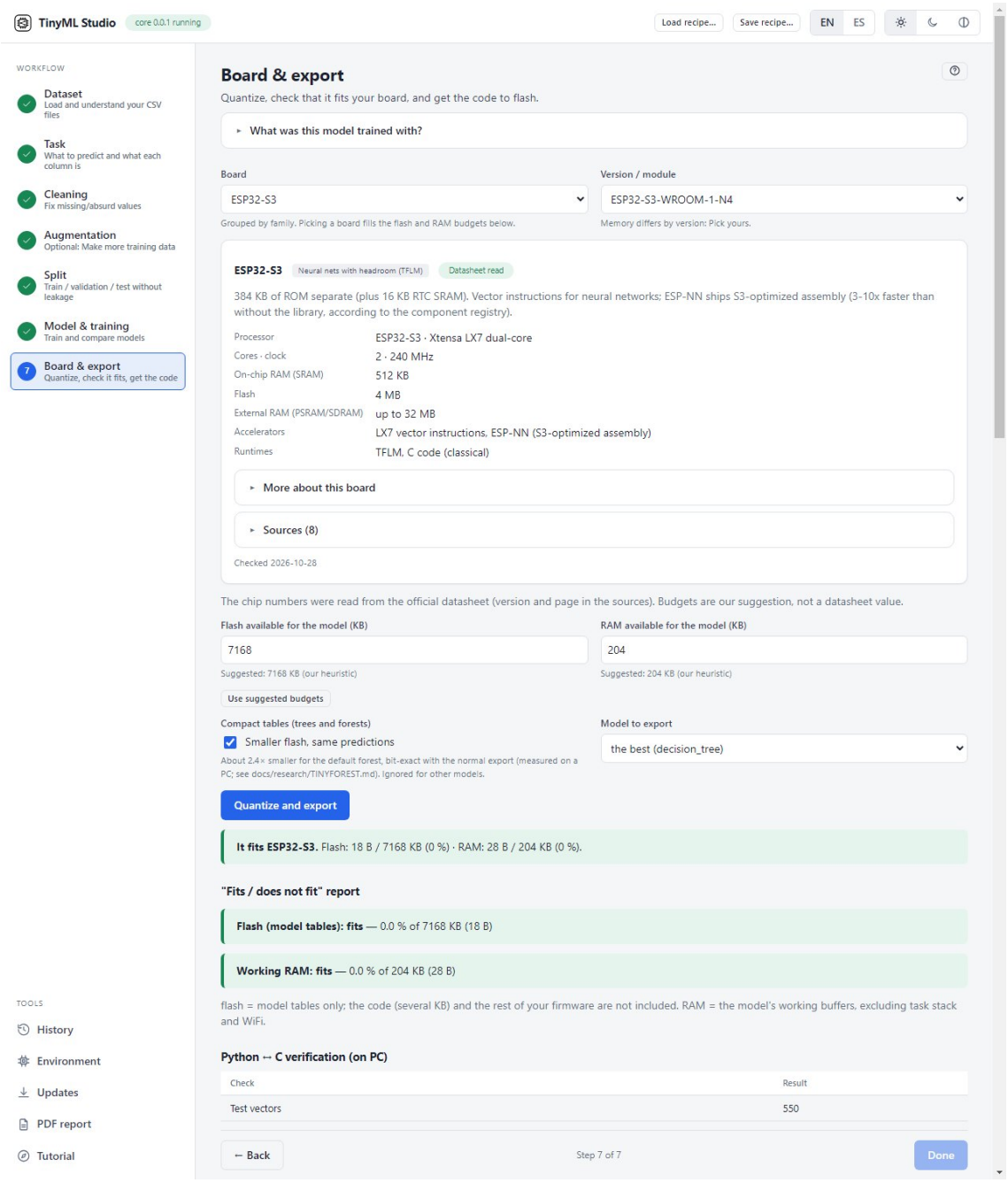


Figure 11. Step 7: the board, the memory budget, the export button and the fit report.

10.1 Pick the board and the budget

The board picker lists the supported boards grouped by family, with variants such as the memory size of the module. A card shows the facts of the chip and where each number comes from. Facts marked as verified were read from official documents. The budgets you can use are our suggestion and not a manufacturer value.

Flash available for the model and RAM available for the model are the amounts you can spend. Use suggested budgets fills them in. If you leave them empty, the report gives the size used but cannot say whether it fits.

> Remember. Your firmware also needs memory for the program, Wi-Fi, sensors and the stack. Leave margin.

10.2 Quantize and export

The button Quantize and export converts the model, checks it, and produces the files. For classical models it writes plain C (tm_model.c and tm_model.h). Compact tables makes trees and forests smaller with the same predictions. For neural networks it writes TensorFlow Lite in float, int8 and float16, and the arrays for TensorFlow Lite Micro.

The fit report shows two verdicts, flash and RAM. Fits, tight and does not fit are shown with the numbers. If it does not fit, the app lists ways out: a smaller model, fewer inputs, or a larger module.

10.3 Verification

A table compares the exported code with the original model on the test vectors: how many agree, the largest difference, and whether the code compiles. This was verified on the computer. The final proof is to run it on your board.

10.4 What was this model trained with?

A card at the top of step 7 lists, in one place, the files and the number of rows, the task, what the model predicts, the input columns, the model, the split, the class balance, the cleaning steps and any new rows from augmentation. Compare it with the data you will measure in real life, using the same units, the same sign and the same columns.

10.5 Try the model with your own values

For classical models, a card lets you type the values of each input and press Predict. It answers with the class and its confidence, or with the value for a regression. It uses the same cleaning and the same model as the exported C code, which was checked equal on the test vectors. It is not a test on the board.

Under each box you see the range that column had in training. If you type a value outside that range, a warning appears, because the model extrapolates there and the answer may be unreliable. An empty box means missing, and the model uses the filler it learned.

The exported C code has the same protection: tm_out_of_range(num) returns how many numeric inputs are outside the training range, and 0 means all fine. Call it in your firmware before trusting a prediction.

10.6 Saving the Arduino sketch

Nothing is saved on your computer until you press Save the Arduino sketch and choose a folder. The app then shows the exact path of the .ino file to open in the Arduino IDE. The button Save files saves everything the export produced.

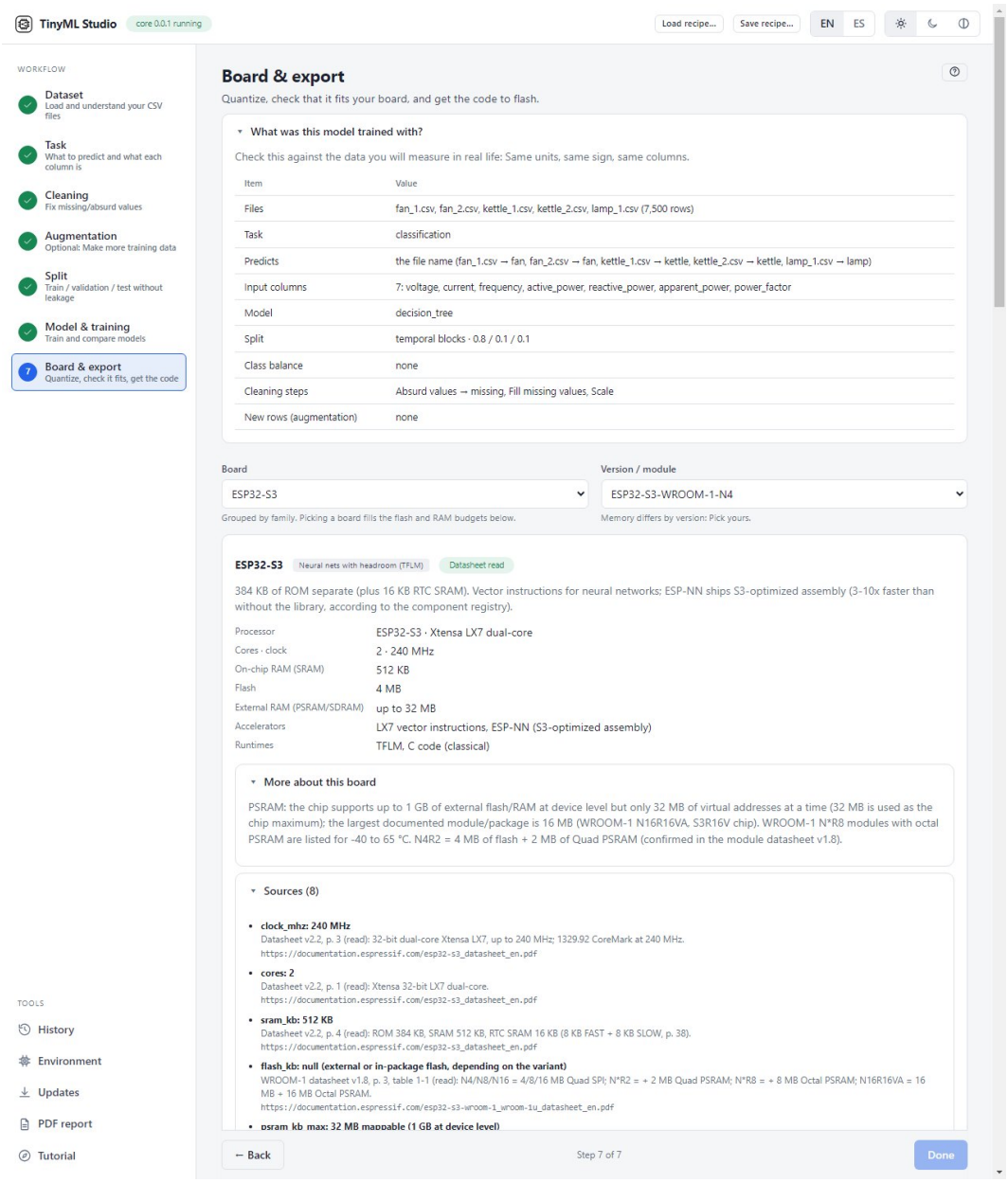


Figure 12. Step 7 with every card expanded: ONNX, which columns matter, speed and unknown answers.

10.7 Extra tools of this step

Export ONNX

Writes a portable model for computers, servers and phones. Cleaning and encoding are inside the model, so it accepts raw values. It was compared with scikit-learn on your test data.

Which columns matter

Shuffles one column at a time and measures how much the result drops. Columns that barely matter could be removed to save sensors and memory. Retrain without them to

confirm.

How fast would it be (estimate)

Counts the operations of the model and multiplies them by assumed cycles per operation and the clock of each board. The result is a range, not a measurement. Time the prediction on your board for the real figure.

Answer unknown when unsure

A classifier always picks one of its classes, even for something it never saw. This sets a confidence threshold and checks it by retraining without one class at a time. The ONNX export can apply it. The C export does not apply it yet.

11. Using the model on an ESP32-S3

This chapter explains the path from the saved files to a running board.

1. Install the Arduino IDE and, in the Boards Manager, the esp32 platform from Espressif Systems.
2. Press Save the Arduino sketch in step 7 and open the .ino file shown. The folder must have the same name as the .ino file.
3. Choose the board ESP32S3 Dev Module, connect the board and press Upload.
4. Open the Serial Monitor at 115200 baud. The example sketch prints pred= and expected=. They must match.
5. Replace the example sample by your own readings and call tm_predict.

11.1 The function tm_predict

tm_predict(num, cat) receives the raw values, in the order printed in the README of the sketch and in the input table of step 7, and returns the answer. For a classifier the answer is the index of the class in the list of class names. For a regression it is the predicted number. The function applies the same cleaning and encoding as the training, so give it raw values with the same units as your CSV files.

When the model has no categorical inputs, pass NULL as the second argument.

> What if the units differ? If the board measures a quantity in different units or with a different scale than the CSV used for training, the model cannot be right. The values must be produced the same way in both places.

11.2 Out of range inputs

A model only knows the range of values it saw in training. Linear models continue the line outside that range, so a value far outside gives an answer that looks confident and is unreliable. Check ranges in your firmware and decide what to do when an input is outside them.

12. Tools: history, reports and the terminal

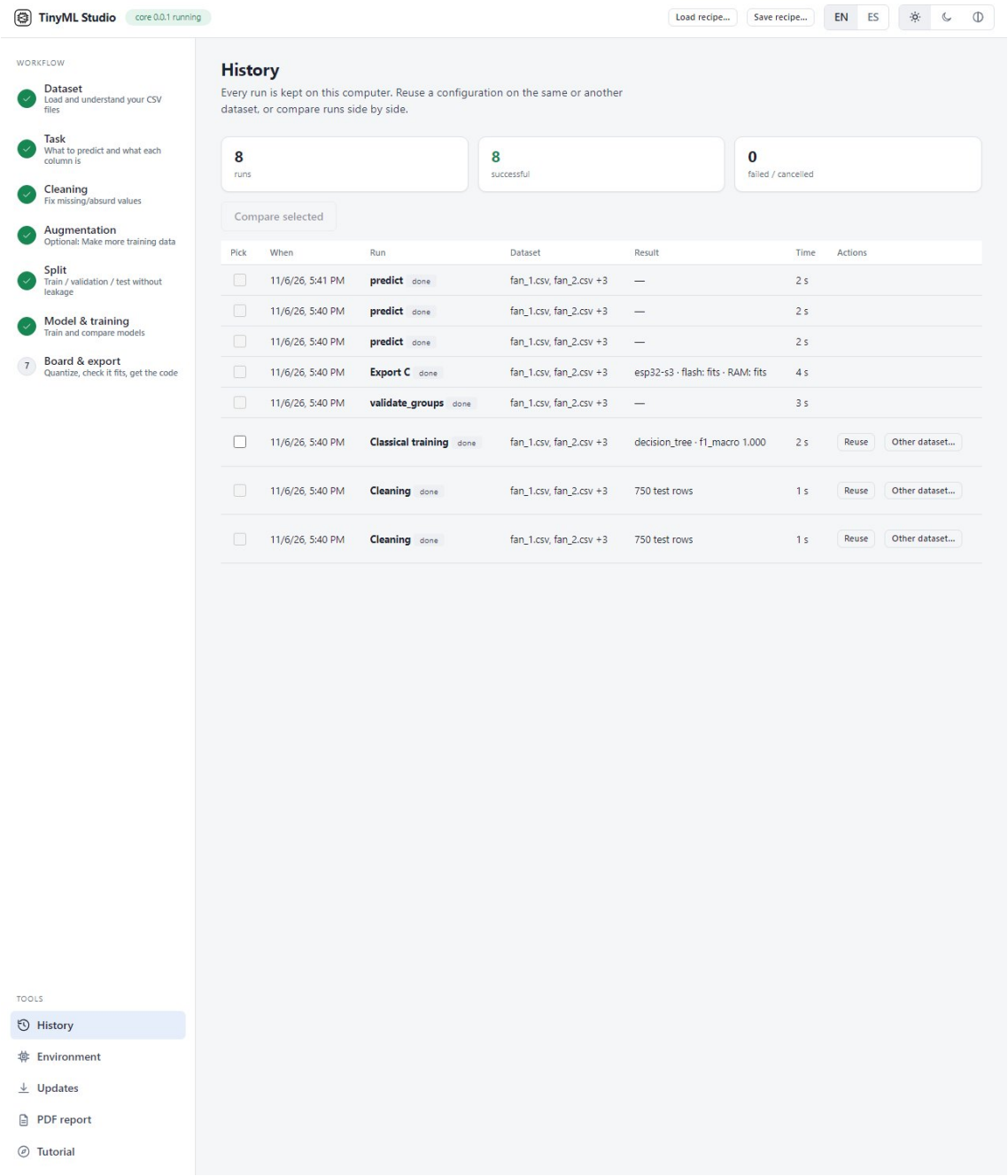


Figure 13. History lists past runs. Open one to compare it or reuse its configuration.

12.1 History

Every training and export is stored. History lets you compare runs, open the files of a past run, and reuse a configuration on a different dataset.

12.2 PDF report

Creates a report with the data summary, the cleaning, the split, the metrics and the charts of the last run, to attach to your documentation.

12.3 Command line and terminal interface

Everything the window does is also available in the terminal. The command tinyml-studio has subcommands such as inspect, clean, train, export, export-onnx, tune, importance, latency, reject and anomaly. Run it with the option help to list them. The terminal interface (tui) is a text mode version of the workflow.

12.4 The guided tour

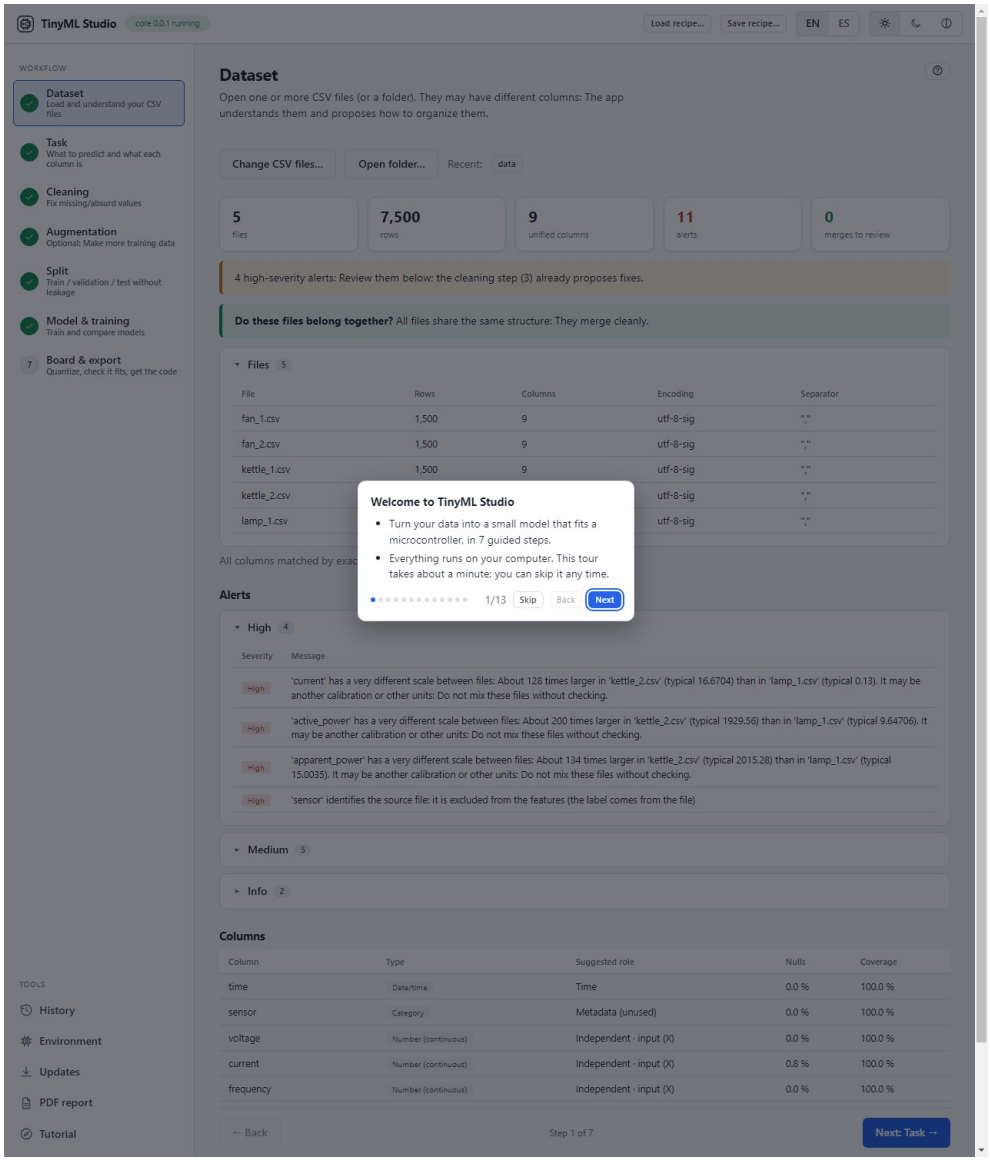


Figure 14. The guided tour highlights each part of the window.

The Tutorial button starts a tour of the window, and every page has a question mark button with tips.

13. Troubleshooting

Problem	Likely cause	What to do
The window asks to set up the app.	No Python environment yet.	Choose option 1 or point to your own environment.
Steps are locked.	The previous step is not complete.	Open a dataset first, then continue in order.
The numbers are suspiciously perfect.	Leakage, or one recording per class.	Remove columns that identify the file. Check Does it work on a new session. Use a split by group.
Training does not use my GPU.	Classical models use the CPU. The environment does not see the GPU.	Read the GPU support card. Install the GPU build of PyTorch or TensorFlow.
The model does not fit.	It is too large for the budget.	Use Settings that fit my board, a smaller model or fewer inputs.
Nothing appears in the Serial Monitor.	Wrong port or speed, or the line ending.	Use 115200 baud. Close other programs that use the port.
Pressing Enter in the monitor does nothing.	The line ending is set to No Line Ending.	Choose Newline, or wait half a second after typing in the example survey.
The board returns strange values.	Inputs in a different order or units.	Follow the input table exactly.

14. What is verified and what is not

This section is deliberately plain. It lists what has been tested and what has not.

14.1 Verified

1. The workflow runs end to end in the desktop application on Windows 11.
2. Exported C code gives the same answers as the Python model on thousands of test vectors, on the computer.
3. A regression model exported to C ran on a real ESP32-S3 and returned the same values as Python, including an interactive survey sketch over the serial port.
4. Training with an NVIDIA GPU through PyTorch on Windows 11.

14.2 Not verified yet

1. Classification models running on a board, and the speed or power use measured on a board.
2. Training on AMD GPUs, and macOS.
3. The installer on a clean Windows account, and the Linux packages on a real Linux system.
4. The Arduino sketches for TensorFlow Lite Micro in the real IDE.

The speed figures of the application are estimates. Board facts that are not marked as verified

come from secondary sources, and every field lists where it came from.

15. Glossary

Anomaly detection
Finding data that does not look like the normal examples, without examples of faults.

Arena
The block of RAM that TensorFlow Lite Micro uses for working values.

Benchmark
Training several models on the same splits to compare them fairly.

Class
One of the categories a classifier can answer.

Cross validation
Repeating training and validation on different parts of the data to measure stability.

Epoch
One pass of a neural network over the whole training set.

Feature importance
A measure of how much each input column matters to a model.

Float16, int8
Smaller number formats. Int8 uses one byte per weight and suits microcontrollers.

Group
A recording or session, used to split and validate without leakage.

Hyperparameter
A setting of a model that you choose before training, such as the depth of a tree.

ONNX
An open file format for models that many programs can run.

Overfitting
A model that memorised the training data and fails on new data.

Quantization
Storing a model with smaller numbers so it uses less memory.

Recipe
The saved configuration of a whole run, as a JSON file.

TensorFlow Lite Micro
A small runtime to run neural networks on microcontrollers.

Threshold
A value above or below which the application takes a decision, such as answering unknown.

16. License and credits

TinyML Studio is free software under the GNU General Public License version 3. You may use, study, modify and redistribute it. Any distributed derivative work must remain open under the same license, and must keep the notice of the original author. This license allows commercial use. It does not allow closing the code.

Copyright (C) 2026 BrianGTRxx. The project was designed and directed by BrianGTRxx and developed with the assistance of Claude, an AI model by Anthropic, used through Claude Code. The author decided the goals, reviewed the results and is responsible for the project.

The screenshots of this manual use synthetic recordings created for the purpose, not real measurements.